

Daily Work Report

Employee Name: Atharva Ravikiran Avhad

Date: 09/08/2026

Reporting Manager: Aditya Chauhan

Work Summary

Successfully implemented the first three days of the Week 4 objective — building an Intelligent Document Processing Pipeline for Retrieval-Augmented Generation (RAG). Extended the existing production-ready PDF extraction microservice with a complete downstream preprocessing pipeline covering document chunking, multi-model embedding, metadata tagging, vector storage, and retrieval endpoints. All components were built additively without breaking any existing Week 3 functionality or test cases.

Completed Tasks

- **Chunking Module (Day 1):** Implemented chunker.py with four distinct strategies — Fixed-Size (tiktoken token-based), Recursive (hierarchical separator splitting), Semantic (cosine-similarity breakpoint detection using sentence-transformers), and Layout-Aware (table-boundary and heading-aware splitting). All four strategies share a common BaseChunker interface, making them swappable via a config flag. Executed a full chunking benchmark across the test PDF dataset, recording chunk count, average/median token size, variance, boundary-violation rate, and wall-clock time per strategy.
- **Embedding Module (Day 2):** Implemented embedder.py supporting three local embedding models — all-MiniLM-L6-v2 (384-dim, fastest), BAAI/bge-small-en-v1.5 (384-dim, higher retrieval quality), and intfloat/e5-base-v2 (768-dim, strongest recall). Each model implements a common BaseEmbedder interface with embed() and embed_query() methods and a built-

in `.benchmark()` function. Finalized the chunk metadata schema as a Pydantic model (`schemas.py`) containing all required fields including `prev_chunk_id/next_chunk_id` for context-window expansion at retrieval time.

- **Pipeline Wiring & Vector Store**

(Day3): Built `rag_pipeline.py` connecting all five stages end-to-end — Extract → Chunk → Embed → Attach Metadata → FAISS Store.

Selected FAISS as the vector store (justified over Chroma for benchmarking purity — no network I/O skew). Added two new API endpoints — POST `/api/v1/ingest` (full pipeline upload) and POST `/api/v1/query` (cosine-similarity retrieval with optional metadata filtering). Wrote `throughput_benchmark.py` to measure documents/minute, chunks/second, and peak memory across a 6-combination strategy × model matrix. All changes pushed to the main branch on GitHub.

Pending Tasks

- Day 4: Build the labeled retrieval evaluation set (15–20 queries with ground-truth chunks).
- Compute Precision@k, Recall@k, MRR, and NDCG@k across the full strategy × model matrix.
- Compile and finalize all four deliverables — preprocessing framework, benchmark report, architecture document, and retrieval quality comparison report.

Plan for Tomorrow

- Build the labeled query evaluation dataset from the test PDF corpus.
- Run the full retrieval quality evaluation across all strategy × model combinations.

- Compile the Week 4 Engineering Report and Architecture Document, including a clear recommendation of the best-performing default configuration.

Employee Declaration

I hereby confirm that the above information accurately represents the work completed by me on the mentioned date.

Employee Signature:*Atharva Avhad*